[1]

Machine Learning – Bike Sharing Data Competition

Arijan Seipi

Bastien Olivier Mutzner

Akshaan Murugesu

Kaggle team: ABA

MSc in Statistics, University of Geneva, Switzerland

Machine Learning – Fall 2024

[1] Image from: https://chattahoochee.whitewaterexpress.com/activity/bike-rentals/

1. Introduction
2. Exploratory Data Analysis
    2.1 Data Structure
    2.2 Missing values
    2.3 Data Analysis
    2.4 Feature engineering
3. Modeling
    3.1 Linear Regression
    3.2 Lasso Regression
    3.3 Decision Trees
    3.3.1 Random Forest
    3.3.2 Gradient Boosting
    3.4 Support Vector Machine
    3.5 Deep Neural Networks
4. Results and Conclusion
5. References

1. Introduction

A rental bike company, Mule, provides Geneva residents with the chance to rent bicycles for a short period of time to move around the city. To obtain bank loans, they need sales forecasts for the next 3 months and asked for our help to fulfill this task. Mule has provided us with a dataset from a similar company in another country. The data contains temporal, economic and meteorological information. Furthermore, Mule has also reliable weather forecasts for the next 3 months, which are needed to predict the number of rentals. We will test ML methods with different complexity and level of interpretability to have the best predictions possible. Mule also wants business insights, such as important features impacting the number of bikes rented.

2. Exploratory Data Analysis

2.1 Data Structure

The train dataset has 15’211 rows and 15 columns. There are different column types:

```
RangeIndex: 15211 entries, 0 to 15210
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Id          15211 non-null  int64
 1   dteday      15180 non-null  object
 2   season      15166 non-null  object
 3   yr          15174 non-null  float64
 4   mnth        15175 non-null  object
 5   hr          15185 non-null  float64
 6   holiday     15179 non-null  object
 7   weekday     15171 non-null  object
 8   workingday  15181 non-null  object
 9   weathersit  15180 non-null  object
 10  temp        15027 non-null  float64
 11  atemp       15030 non-null  float64
 12  hum         15054 non-null  float64
 13  windspeed   15047 non-null  float64
 14  cnt         15211 non-null  int64
dtypes: float64(6), int64(2), object(7)
```

- **Id**: ID for each observation
- **dteday**: date
- **season**: season
- **yr**: year
- **mnth**: month

- **hr**: hour of the day
- **holiday**: whether day is holiday or not
- **weekday**: day of the week
- **workingday**: if day is neither holiday nor weekend
- **weathersit**: weather situation
- **temp**: normalized temperature in Celsius. The values are divided to 41 (max)
- **atemp**: normalized feeling temperature in Celsius. The values are divided to 50 (max).
- **hum**: normalized humidity. The values are divided to 100 (max).
- **windspeed**: normalized wind speed. The values are divided to 67 (max).
- **cnt**: count of total rental bikes including both casual and registered.[2]

	temp	atemp	hum	windspeed	cnt
count	15027.000000	15030.000000	15054.000000	15047.000000	15211.000000
mean	0.512780	0.489149	0.624780	0.191712	185.225232
std	0.195879	0.174807	0.195817	0.122418	177.918222
min	0.020000	0.000000	0.000000	0.000000	1.000000
25%	0.340000	0.333300	0.470000	0.104500	39.000000
50%	0.540000	0.500000	0.620000	0.194000	138.000000
75%	0.660000	0.621200	0.780000	0.253700	275.000000
max	1.000000	1.000000	1.000000	0.850700	977.000000

Before going further, we check whether there are outliers among numerical features. We check the minimum and the maximum to see if they are within a plausible range. The column **Id** (int64) is irrelevant for our analysis. The column **yr** and **hr** (float64) will be considered as categorical features. Otherwise, the above figure shows a reasonable range of values for each column. Weather features are normalized, that is why they are between 0 and 1.

2.2 Missing values

There are no missing values in the test set. EDA will be mainly conducted on the training set, because the response variable **cnt** is not available on the test set. There are missing values in each column apart from **Id** and **cnt**. Before changing certain columns into categories, the goal is to fill the missing values if possible.

- **dteday**: we check whether observations having missing values in this column have also missing values in **hr** and they don't. Since it is hourly observations, if the

[2] https://www.kaggle.com/competitions/ml-unige-fall-2024-bike-sharing/data

hour is midnight, we pick up the date of the following observation, otherwise we take from the previous one.

- **season**: after looking at the data and checking that there aren't season-changing dates, we apply the forward fill method.
- **yr**: after checking whether there are year-changing dates, we apply the forward fill method.
- **mnth**: after checking whether there are month-changing dates, we apply the forward fill method.
- **hr**: if the hour of the previous observation is 23, we fill with 0, otherwise we increment by 1 the previous observation.
- **holiday**: since it is hourly observations, after checking whether there are observations where the day changes, we apply a forward fill method.
- **weekday**: because none of the observations with NA values has an hour equal to 0, we apply a forward fill method.
- **workingday**: if the weekday of the observations is a working day, we apply with "Yes", otherwise "No".
- **weathersit**: since the weather does not change drastically from one hour to another, we decide to apply a forward fill method.
- **temp**, **atemp**, **hum**, **windspeed**: we create a general function where it finds the nearest non-NA previous value and the nearest non-NA following value and average them to fill the missing values.

After applying everything, there is still one missing value for **hum** and it is the first observation, which does not have a previous value. We decide to drop it and as our dataset is large enough, it will not impact our predictions. At the end of the filling process, the dataset has 15'210 rows and still 15 columns.

We will now proceed to the type selection of each column, the reordering of each category and their encoding to have a cleaner and less dense dataset. We drop the column **Id** and **dteday**, since most of the information is already in the other temporal features.

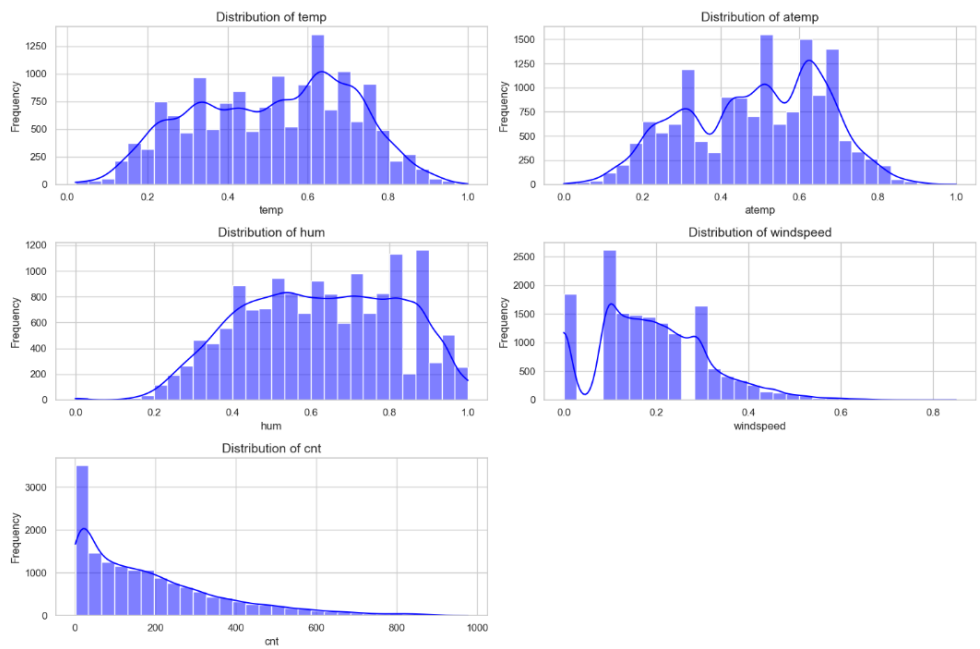
The numerical features are **temp**, **atemp**, **hum**, **windspeed**. They are continuous measurement.

The categorical features are **season**, **yr**, **mnth**, **hr**, **holiday**, **workingday**, **weathersit**.

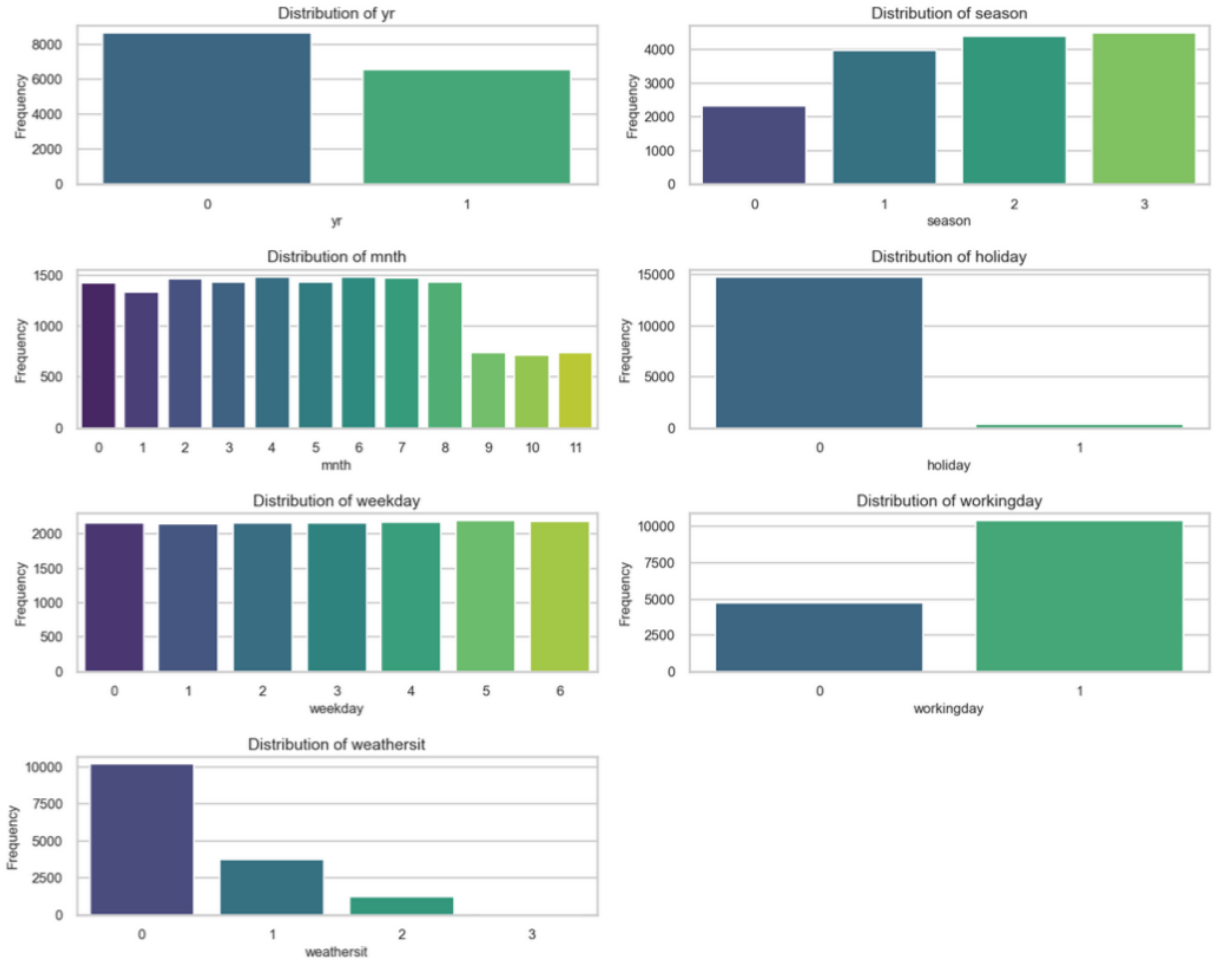
- **season**: 0 for Fall, 1 for Winter, 2 for Spring, 3 for Summer.
- **yr**: 0 for 2011 and 1 for 2012.
- **mnth**: 0 for January up to 11 for December.
- **hr**: they are already encoded in the right order. We transform it as a category.

- **holiday**: 0 for No, 1 for Yes.
- **weekday**: 0 for Monday up to 6 for Sunday.
- **workingday**: 0 for No, 1 for Yes.
- **weathersit**: 0 for "Clear, Few clouds, Partly cloudy, Partly cloudy",
  1 for "Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist",
  2 for "Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds",
  3 for "Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog".

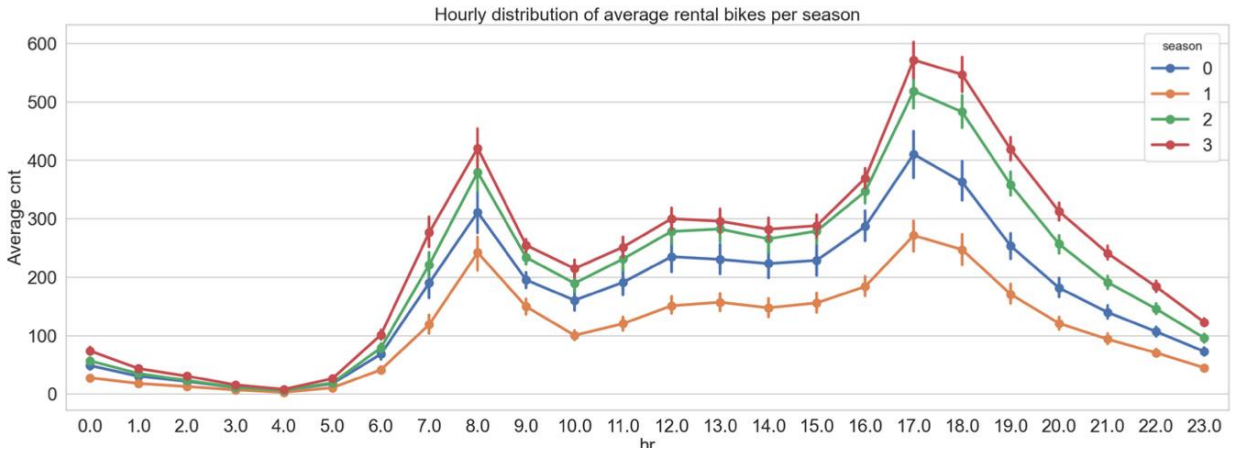
2.3 Data Analysis



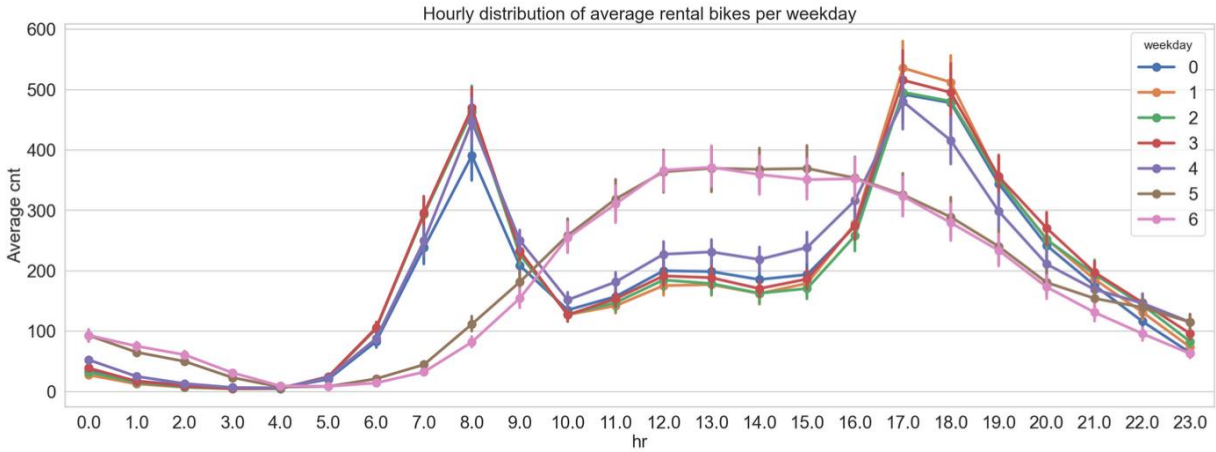
The first four plots show the number of observations for each numerical feature. The distribution of normalized temperature and feeling temperature are both slightly left-skewed. There are more observations available when the temperature is higher. The distribution of humidity looks like a slightly right-shifted uniform distribution. Concerning windspeed, there are less observations as the weather gets worse, suggesting that extreme weathers are rare. The missing bars are present because the dataset doesn't contain any windspeed of that value, so we don't have observations for it. Finally, the distribution of bike rentals counts is heavily right skewed: high values have less probability to happen.



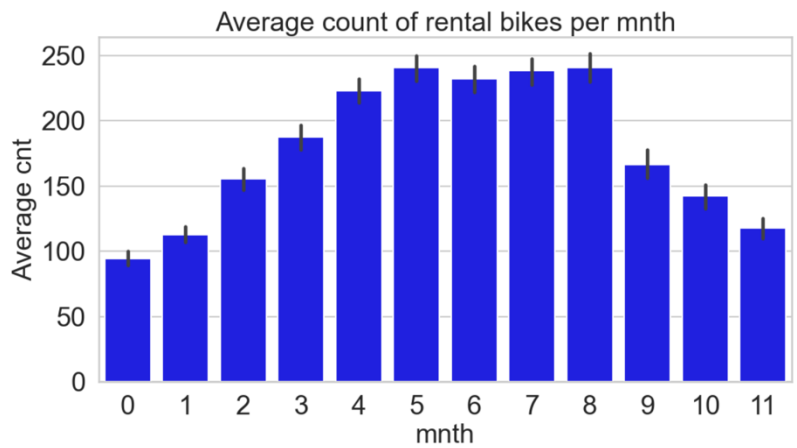
Concerning categorical features, above plots are the number of observations for each category. This helps to see if the latter is balanced. There are less observations in year 2012 (1), since a part was taken as the test dataset by the teaching assistant. Otherwise, observations are balanced between these two years. Concerning season, there are less observations in Fall (0) suggesting only a slight imbalance in data. Again, there are less observations for October, November and December as it will be the test data provided by the assistant. Otherwise, observations seem to be balanced through the months. Concerning holidays, there is a significant imbalance: there are only a few observations when it's a holiday (1). Observations are balanced through the weekdays. In working day, Observations are imbalanced as there are more working days (1) than weekends and holidays (0). Finally, the worse the weather situation gets, the less observations we have. It suggests that extreme weather does not happen often. Let's see the average counts of rental bikes for these categorical features.



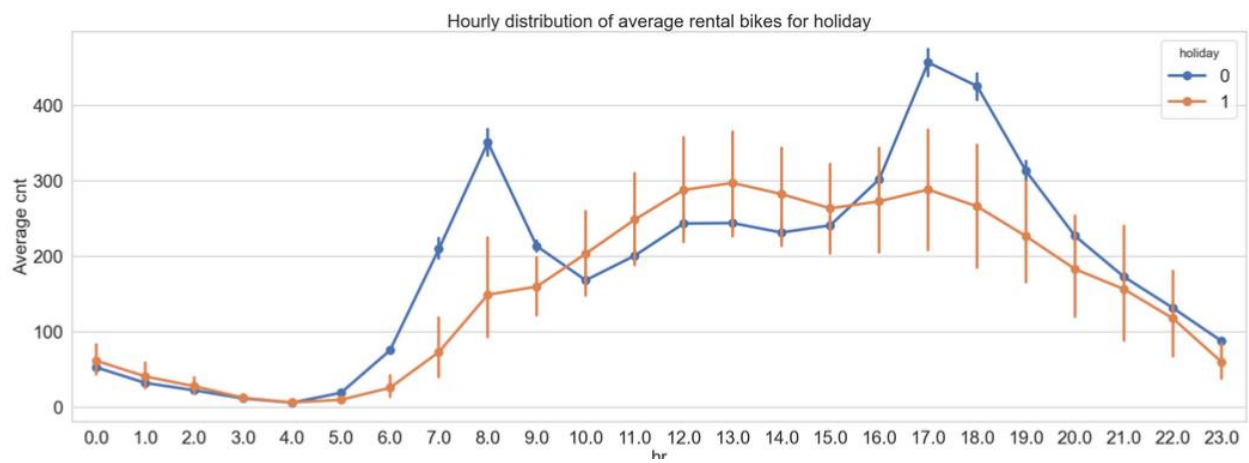
This plot shows the hourly distribution of average counts for each season. Winter (orange) has the lowest counts, whereas Summer (red) has the highest. It emphasizes the positive correlation between temperature and counts, since it is warmer in Summer. In general, the graph shows a stable use of bikes throughout the seasons. Furthermore, this shows peaks of counts for interval of hours: between 7:00 and 9:00 and between 16:00 and 17:00. This can be linked to people going to work or to school and coming back from it. The demand for bikes seems to be higher during rush hours.



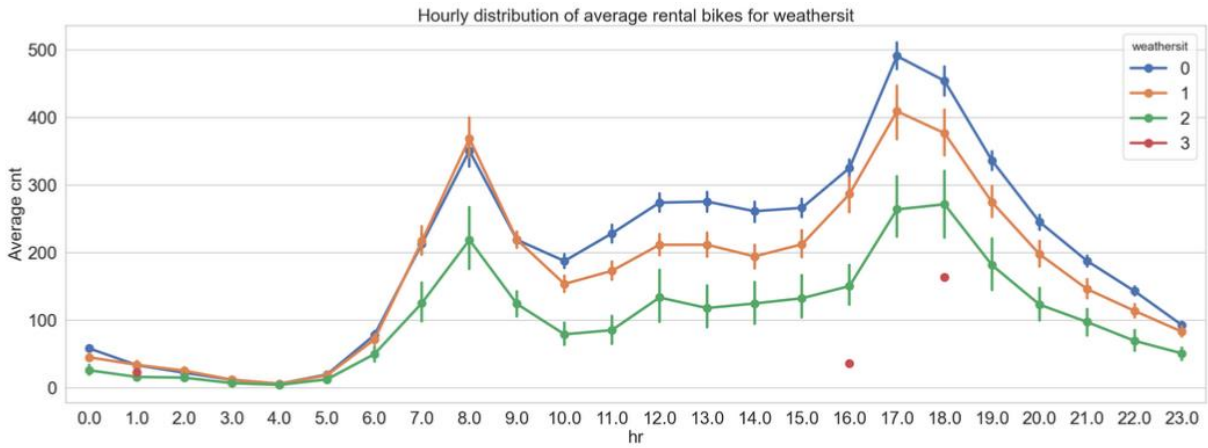
This plot is the hourly distribution of average counts specific to weekday. 0 to 4 are the working days. Their lines show the same pattern as the previous one, emphasizing higher demand for bikes during rush hours. However, this graph brings additional information on bike rentals during weekends (5 and 6) having a different trend. Higher use of bikes comes progressively during the afternoon and declines at the end of the day. There are no peaks. This shows a different type of use during the weekends.



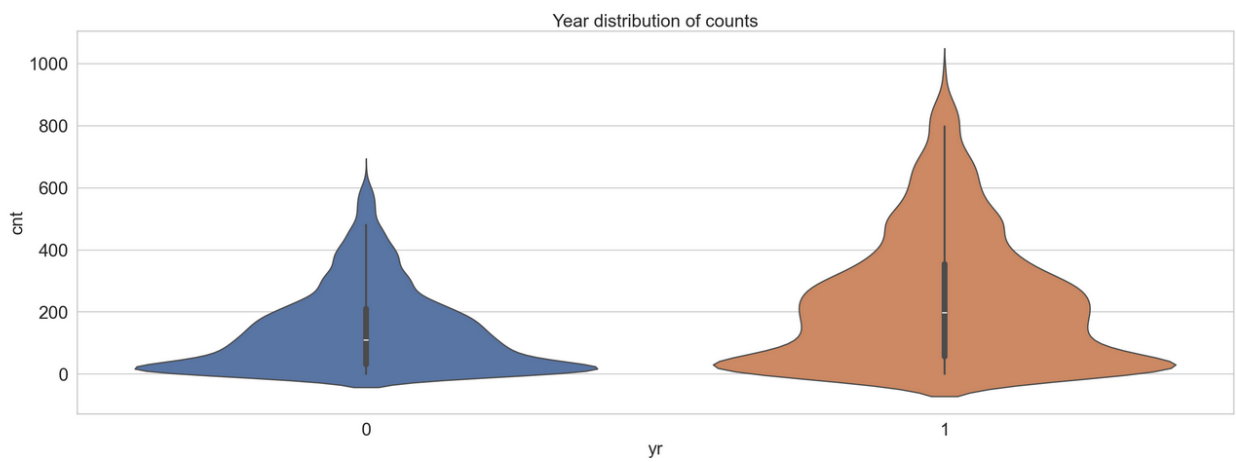
The above figure shows the monthly distribution of average numbers of rental bikes. From June to September (5 to 8), there is a higher use of bicycles. This is closely related to the summer season and higher temperatures, influencing people to rent a bike. As in the other graphs, there is a clear monthly temporal pattern.



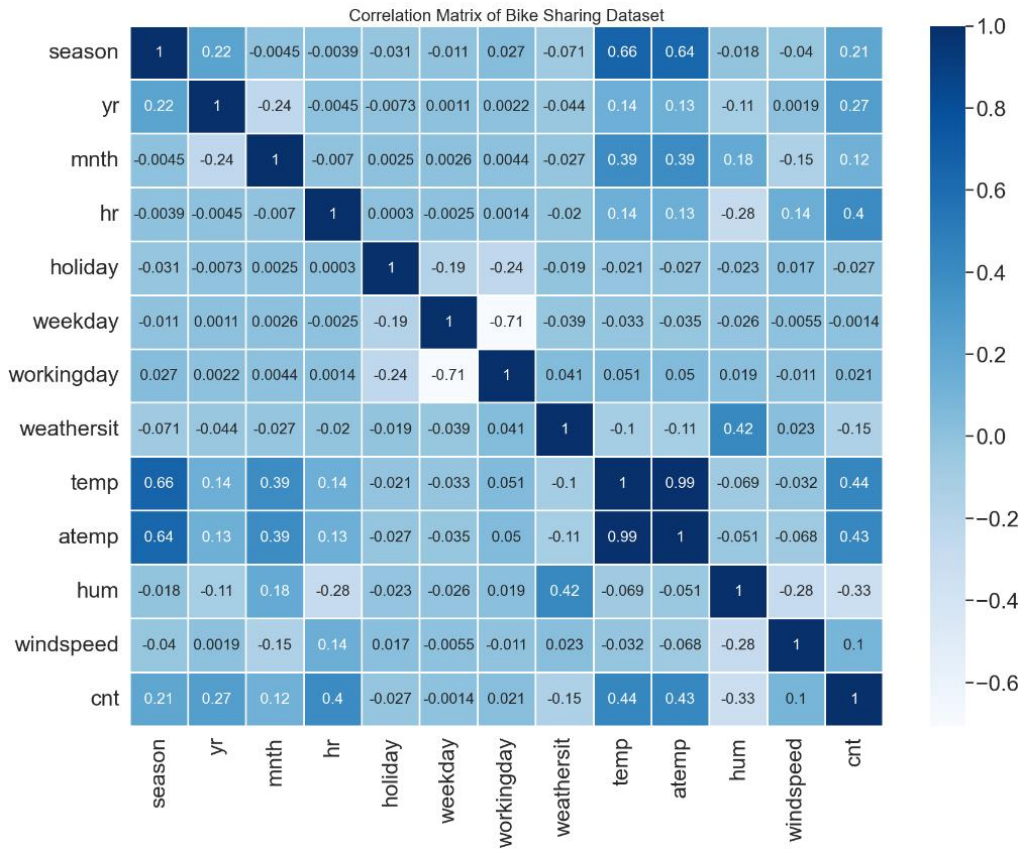
This plot is the hourly distribution of average rental bikes for holiday. During holidays (orange), the use of rental bikes follows the same pattern as for the weekends, without any peak. Since we have less observations for holidays, the confidence intervals for the orange points are larger.



Concerning weather situations, one can see that as it becomes worse, there is less use of rental bikes. In addition to that, this graph emphasizes the high imbalance of observations in this category, especially for extreme weather (3), there are only 3 observations, making the plot of a line impossible. This imbalance can also be noticed in the width of the confidence intervals being larger for light bad weather.



For year, we decided to do a violin plot to see how the counts vary between 2011 (0) and 2012 (1). In the latter, higher values seem to be more likely to happen than in the former. This suggests an increase in demand for rental bikes in 2012. Both have high probabilities in lower values which are consistent with previous graphs where the use of bikes stays low out of the rush hours apart from for the weekends and holidays.



The above figure is the correlation matrix between the columns of the dataset, excluding **Id.** Values vary from –1 (strong negative correlation) to +1 (strong positive correlation). The response variable **cnt** has a positive correlation with weather features like **temp** (0.44), and **atemp** (0.44), and negative correlation with **hum** (-0.33) and **weathersit** (-0.15) emphasizing previous suggestions. Concerning temporal features, the output variable has a positive correlation with **season** (0.21), **yr** (0.27), **mnth** (0.12) and **hr** (0.4), emphasizing the influence of time on the rental bikes counts.

The column **temp** and **atemp** are strongly correlated (0.99), since they contain almost the same information. The temporal features have a positive correlation with **temp/atemp,** since temperature is closely related to seasons, months and hour of the day.

2.4 Feature engineering

During EDA, we have already dropped **Id** and **dteday**. As **temp** is highly correlated with **atemp**, we decide to drop the latter to avoid multicollinearity in linear models. Concerning

tree-based models, redundant features having almost the same information increase the complexity of the model. As we have learned on the course, it is better to have a more parsimonious model, avoiding overfitting as much as possible. Therefore, having only **temp** in the dataset ensures unique and non-redundant information.

Since temporal features have an influence on rental bikes count, it is important to capture their trend and seasonality. The columns **hr**, **weekday** and **mnth** have a circular nature in the dataset: they repeat themselves many times. The hour resets to 00:00 after 23:00. The weekday resets to Monday after Sunday. The month resets to January after December. Sine and cosine transform these temporal variables to capture their cycle (see code) in terms of direction and magnitude. We don't transform **season** because, after our EDA, it seems that **mnth** is already capturing a lot of information concerning the former and we don't to have too many features in our model.

3. Modelling

3.1 Linear Regression

As a foundational and interpretable approach, we decided to implement a linear regression as the initial benchmark model for forecasting bike rentals. Since our objective is to provide accurate predictions while offering business insights, linear regression serves as a suitable starting point due to its simplicity and ability to quantify the relationship between predictors and the target variable.

The model was trained using the processed dataset, which incorporated engineered features like sine and cosine transformations for temporal variables to capture cyclical patterns. Linear regression demonstrated moderate predictive performance, achieving a Mean Absolute Error (MAE) of 91.08 on the test set and 91.28 on the train data. While this MAE is suboptimal compared to more complex models, it highlights linear regression's limitations in handling non-linear relationships within the dataset.

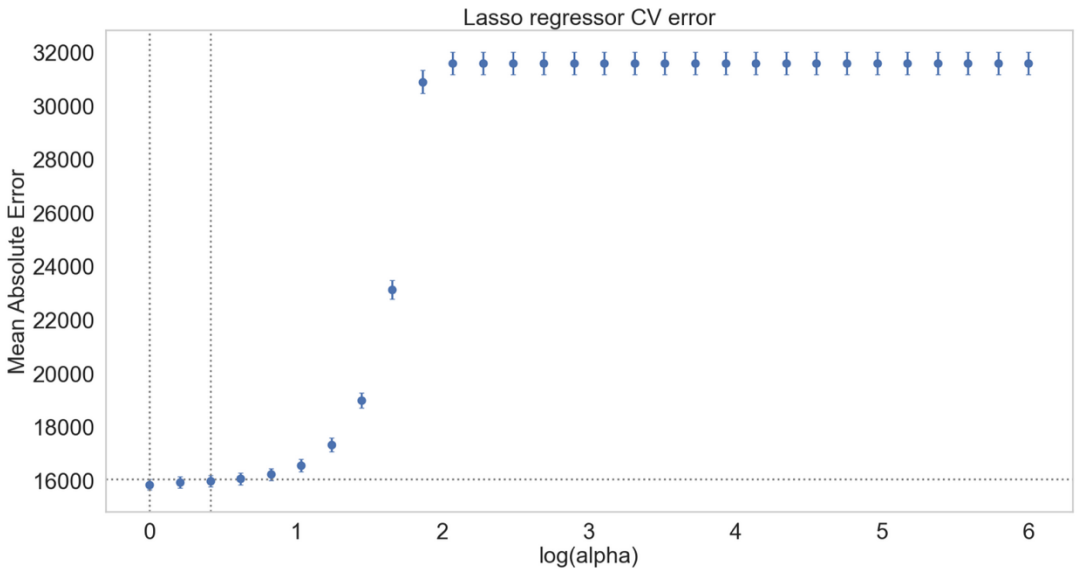
Moreover, the feature coefficients provide valuable interpretability, revealing the importance of variables like normalized temperature (**temp**), year (**yr**), and working day status (**workingday**) in influencing bike rentals. These insights align with the intuition, as warmer weather and weekdays typically see higher demand. However, the model struggles to capture complex interactions, and non-linear dependencies present in the data.

Despite its limited predictive power, linear regression offers a clear baseline for comparison with more sophisticated models like decision trees and neural networks. Its results

underscore the need for more advanced techniques to enhance accuracy while maintaining interpretability for actionable business insights.

3.2 Lasso Regression

Lasso regression was employed to enhance prediction accuracy while simultaneously performing feature selection by incorporating an L1 penalty into the loss function. This penalty forces some coefficients to shrink to zero, effectively identifying the most relevant predictors. We opted for Lasso instead of Ridge regression because usually they both have similar predictive power, but Lasso results are more interpretable[3]. The model was fine-tuned by performing a grid search with cross-validation to determine the optimal regularization parameter alpha, following the One Standard Error (1-SE) rule to balance predictive performance and model simplicity, decreasing the alpha makes the model more complex and similar to standard linear regression. This resulted in a test Mean Absolute Error (MAE) of 91.03 and a train MAE of 91.23, so very similar to the linear regression because of our alpha.

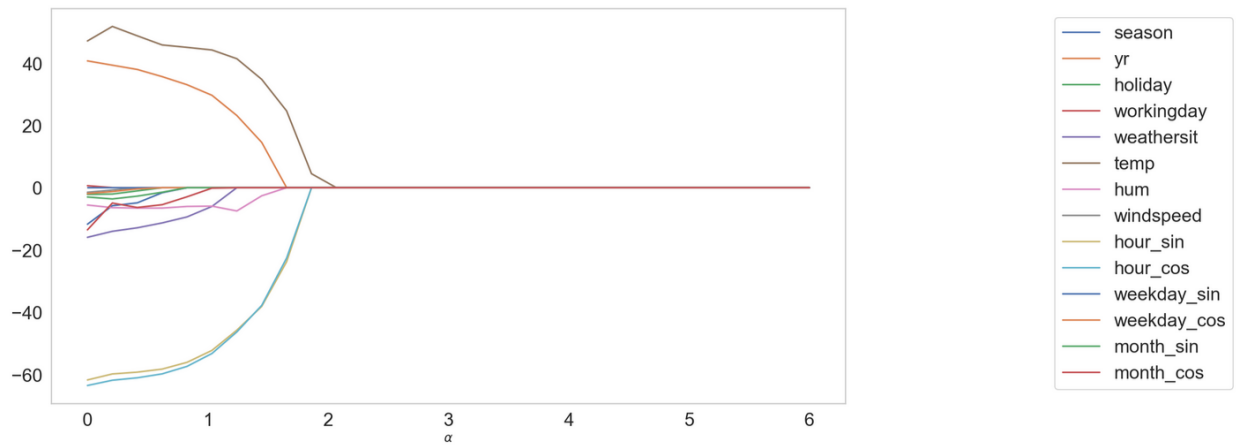


The Lasso regression model retained several key features, such as **season**, **yr**, **holiday**, **weathersit**, **temp**, **hum**, **hour_sin**, **hour_cos**, **weekday_cos**, **month_sin**, and **month_cos**, which were assigned non-zero coefficients. These features were identified as important predictors for bike rentals. For instance, **temp** demonstrated a strong positive influence on

[3] *Machine Learning (S403011): 08. Linear Model Selection, slide 9.*

rental counts, reflecting the tendency for bike usage to increase in warmer weather. The sinusoidal transformations of time-related variables, such as **hour_sin** and **hour_cos**, effectively captured the cyclical nature of hourly rental patterns, aligning with peak times during morning and evening commutes.

On the other hand, certain variables, including **workingday**, **windspeed**, and **weekday_sin**, were reduced to zero by the Lasso penalty. This indicated that these features contributed minimally to the predictive power of the model, either due to their redundancy or weak association with the target variable.



Overall, Lasso regression provided a robust and interpretable model, highlighting the most impactful features while discarding less relevant ones. Although its predictive performance was moderate compared to more complex models, Lasso offered valuable insights into the relationships between predictors and bike rental counts, making it a useful tool for both forecasting and understanding the drivers of demand.

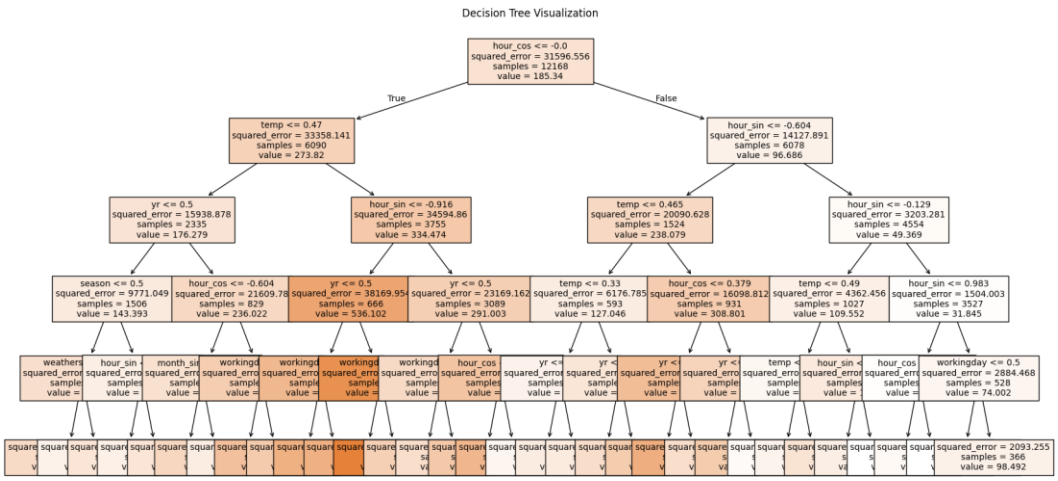
3.3 Decision trees

Decision trees are a class of predictive models that stratify or segment the predictor space recursively into simpler regions. For a given observation, predictions are made by taking the average of the training observations within the region to which the observation belongs. This approach is intuitive and provides a clear structure for understanding the model's decision process.

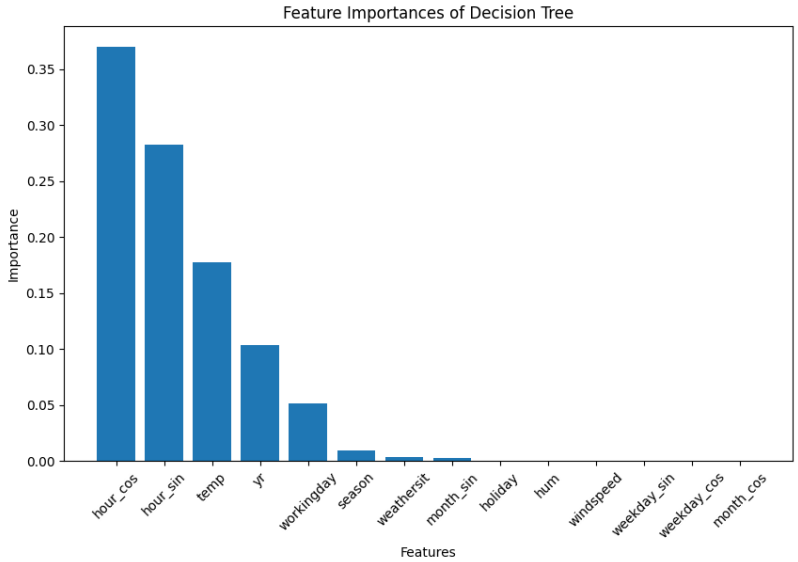
The main categories of tree-based methods are:

1. Decision trees: A single tree that offers good interpretability and transparency.
2. Bagging/Boosting: Methods that combine multiple trees to improve predictive performance.
3. Random Forest: An ensemble of bagged decision trees that are decorrelated to reduce variance.

A Single Tree with a maximum depth of 5 was trained to balance complexity and generalization. The model's performance was evaluated using Mean Absolute Error (MAE), and feature importance rankings were derived to identify significant predictors.



The Single Tree achieved a test MAE of 63.84 and a train MAE of 60.76, showing moderate accuracy compared to ensemble methods. The most influential features included **hour_cos** (importance: 0.3698) and **hour_sin** (0.2823), which highlight peak rental hours during commuting times. **temp** (0.1773) showed a strong positive correlation with rentals, while **yr** (0.1039) and **workingday** (0.0516) captured yearly trends and workday effects. Features such as **windspeed**, **holiday**, and **month_cos** had negligible impact, contributing little to the model's predictions.



The results emphasize the importance of temporal patterns, with **hour_cos** and **hour_sin** dominating the feature importance rankings, underscoring the need to optimize bike availability during commuting hours. Warmer temperatures were shown to increase demand, supporting seasonal inventory planning, while yearly trends suggested potential for business growth. However, the model's limitations lie in its simplicity; deeper trees risk overfitting the training data. For this reason, we constrained the tree's depth to avoid overfitting and maintain interpretability.

Cost-complexity pruning offers an approach to refine the model by evaluating splits based on their contribution to prediction error reduction and pruning those with minimal impact. While this technique could balance simplicity and accuracy, we chose not to apply it here, as the primary goal was interpretability rather than pure performance. Instead, we focused on limiting the tree's depth and relied on ensemble methods like Random Forests and Gradient Boosting for more accurate predictions. These methods, though less interpretable, demonstrated superior performance for forecasting bike rentals. The trade-off between interpretability and performance guided our decision to maintain the Single Tree's simplicity while leveraging ensemble methods for higher accuracy.

3.3.1 Random Forest

Given the limitations of single decision trees, such as their tendency to overfit the data, we opted to use a random forest regressor for our bike-sharing dataset analysis. The random forest method mitigates overfitting by combining multiple decision trees, each trained on a bootstrap sample of the data, and averaging their predictions. This ensemble approach

reduces variance and improves the model's robustness. Furthermore, in bagging, the trees grown with each bootstrap data set share many common training observations and the greedy algorithm then chooses often the same predictors for early splits, meaning the trees are very similar and the predictions very correlated. Whereas the idea of Random Forest is to decorrelate the trees from the bootstrap data sets by introducing constraints in the growing process. The constraint is to select a random subset of m variables for each split to avoid having too similar trees. From our course of Machine Learning, it is said that p/3 typically works well in practice, so we will choose this value and search around it during cross-validation to optimize the model.

The random forest is particularly well-suited for this task because it can capture complex, non-linear relationships between predictors without requiring extensive preprocessing. One of the drawbacks is that we can no longer plot a single tree for interpretation, but it provides a measure of feature importance, allowing us to identify which factors most influence the number of bike rentals. The random forest's ability to handle large datasets and its resistance to noise make it a practical choice for forecasting tasks.

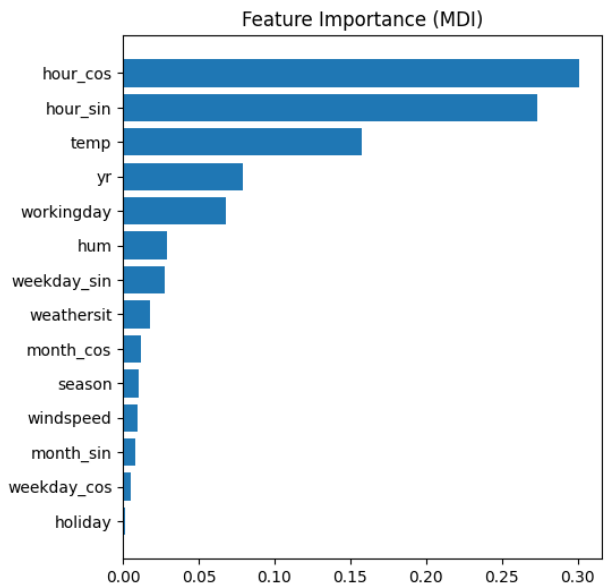
We implemented the Random Forest Regressor to predict bike rentals, focusing on minimizing the Mean Absolute Error (MAE) to align with the competition's evaluation criteria. To optimize the model, we conducted a grid search with 5-fold cross-validation, testing a range of values for the hyperparameters: **n_estimators** (number of trees), **max_depth** (maximum depth of the trees), and **max_features** (number of features considered at each split).

The grid search identified an optimal configuration with 300 estimators, a maximum depth of 18, and five features considered for each split. This configuration delivered strong predictive performance, achieving a test MAE of 25.91 and a train MAE of 9.74, which indicates the model's precision in forecasting bike rentals. The validation MAE on the leaderboard was higher at 54.37, reflecting the model's robustness in generalizing beyond the training data. In conclusion, Random Forest have good predictive power in this context, but we lose interpretability.

We chose not to refine the grid search further because Random Forests are known for their robustness and generally perform well even with heuristic hyperparameter values. As highlighted in our Machine Learning course, these models require almost no tuning because their out-of-bag error estimates provide a good indication of generalization error without the need for extensive cross-validation. Additionally, the computational cost of further refinement was not justified by the potential marginal gains in performance. Moreover, while Random Forest achieved good results, we observed that significantly better performance

was obtained with more complex models. These factors made additional optimization of the Random Forest model unnecessary.

To further assess the model, we examined the feature importance scores using the Mean Decrease Impurity (MDI) metric. This analysis revealed that the most influential predictors were **hour_cos** and **hour_sin**, followed by **temp** and **yr**, indicating that temporal and seasonal variations are key factors driving bike rental patterns. This insight underscores the model's ability to identify and leverage complex, non-linear relationships between the predictors and the target variable, even if some interpretability is sacrificed compared to simpler models.



3.3.2 Gradient Boosting

In our analysis of the bike-sharing dataset, we also employed the Gradient Boosting regression due to its effectiveness in capturing non-linear relationships and reducing bias through iterative learning. Gradient Boosting builds an ensemble of decision trees sequentially, with each tree focusing on correcting the errors of the preceding model, so using information on previously grown trees, which is the main difference with bagging and random forest. This makes it particularly suited for tasks where intricate patterns in the data need to be captured. However, its sequential nature also increases the risk of overfitting, necessitating careful hyperparameter tuning. The tree tuning parameters to focus on, based

on the machine learning course, are the number of trees (overfitting!), the learning rate and the number of splits.

So, we performed a grid search with 5-fold cross-validation specifically focusing on those hyperparameters: the number of estimators (500, 700, 1000), learning rate (0.01, 0.05, 0.1), and maximum tree depth (1, 3, 5), searching around the values that usually works in practice as per the course ML. The grid search identified the best hyperparameters as 1000 estimators, a learning rate of 0.1, and a maximum depth of 5. With these settings, the model achieved a test MAE of 24.32. However, the MAE on the Kaggle leaderboard dataset remained high at 42.39, indicating that while the model's performance improved on the test set, its ability to generalize to unseen data was limited but still improved.

To balance performance and model simplicity, we applied the One Standard Error (1-SE) ru le, selecting the simplest model whose performance was within one standard error of the b est model. This approach led to a model with 700 estimators, a reduced learning rate of 0.0 5, and a maximum depth of 5. Despite being simpler, this model achieved a comparable te st MAE of 25.24 and a leaderboard MAE on Kaggle of 43.29, suggesting that reducing the lea rning rate helped mitigate overfitting but sacrificing a bit performance.

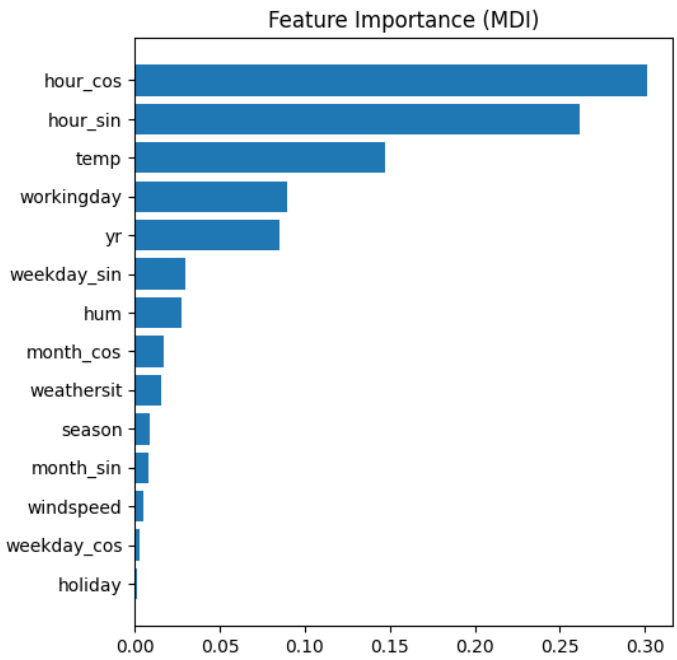
To refine the model further, we conducted an additional grid search focused on a narrower range of hyperparameters, around the last values for each hyperparameter obtained. Specifically, we tested 800, 1000, and 1200 estimators, learning rates of 0.08, 0.1, and 0.12, while keeping the maximum depth fixed at 5. The best-performing model from this refined search used 800 estimators and a learning rate of 0.08, achieving a test MAE of 23.99 and s train MAE of 12.54. However, when tested on the Kaggle leaderboard, the MAE increased to 43.32, indicating that the model’s generalization performance did not improve and, in fact, slightly deteriorated.

These results highlight a consistent trade-off between model complexity and generalization. While gradient boosting models produced excellent results on the test set, their performance on the leaderboard dataset did not improve significantly and, in some cases, worsened. This suggests that while gradient boosting effectively captures patterns in the training data, it remains prone to overfitting, especially when the learning rate is high, or the number of estimators increases.

Model Configuration	Learning Rate	Estimators	Max Depth	Test MAE	Leaderboard MAE
Initial Model	0.1	1000	5	24.32	42.39
Best Grid Search Model	0.05	700	5	25.24	43.29
Simpler Model (1-SE Rule)	0.08	800	5	23.99	43.32

[4]

In conclusion, Gradient boosting provided strong predictive performance on the test set, but the higher error on the leaderboard dataset underscores the importance of balancing model complexity and generalization. The results suggest that simpler models, such as those identified through the 1-SE rule, may offer a more robust solution by maintaining good accuracy while reducing the risk of overfitting.



The feature importance plot reveals that temporal and weather-related factors are the primary drivers of bike rentals. The engineered features **hour_cos** and **hour_sin**, representing the time of day, significantly influence the model, capturing peak rental patterns during commuting hours. Temperature also plays a crucial role, reflecting the strong impact of weather conditions on bike usage. The distinction between weekdays and

[4] *The style and format of the table was generated with the assistance of ChatGPT. The data and values presented in the table were independently provided and are not derived from ChatGPT.*

weekends, captured by the **workingday** feature, further highlights the importance of temporal trends.

Secondary contributors include **yr**, indicating yearly trends, and **weekday_sin** and **weekday_cos**, which capture weekly cycles. The feature **hum** has a moderate impact. Features like **holiday**, **weathersit**, and **month_cos** have minimal influence, possibly due to redundancy or lower variability in the dataset.

These findings suggest that focusing on peak hours, favorable weather, and weekday patterns can optimize bike-sharing operations. The importance of engineered features demonstrates the value of preprocessing, aligning with the project's goals of extracting actionable insights from key predictors.

3.4 Support Vector Machine

Support Vector Machine Regression (SVR) is a flexible approach for capturing nonlinear relationships, using kernel functions like the radial basis function (RBF) to map data into higher-dimensional spaces. Its ε-insensitive loss focuses on overall trends, ignoring noise, and its hyperparameters (C, ϵ, and γ) allow fine-tuning to balance complexity and generalization.

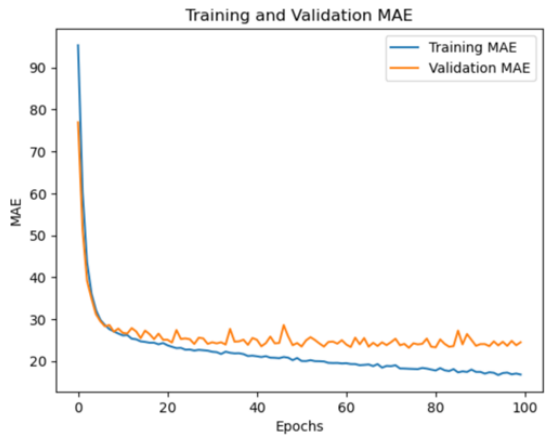
In the context of this bike-sharing prediction, SVR is a suitable candidate for several reasons. First, the dataset contains temporal, meteorological, and economic features that are likely to interact in nonlinear ways. SVR, with its kernel functions, can capture such relationships effectively. Then, the ε-insensitive loss function ensures that the model focuses on data trends while ignoring minor variations or noise, which may exist in real-world data. Given these advantages, we aim to explore SVR as an alternative to tree-based models like Random Forest and Gradient Boosting. This will provide insights into whether kernel-based regression methods can complement or outperform ensemble methods in this context.

Applied to the bike-sharing dataset, SVR was evaluated after feature standardization and hyperparameter optimization via grid search. The best model achieved a test MAE of 43.22 with parameters C=700, ϵ=0.7, γ="auto," and the RBF kernel. However, SVR underperformed compared to Random Forest and Gradient Boosting, which achieved MAEs below 30, highlighting SVR's sensitivity to parameter tuning and its challenges in generalizing complex feature interactions. Furthermore, it’s Kaggle Leaderboard MAE with this model was 65.85, so preforming. Given these results, we did not focus extensively on this model, especially considering its computational time and the challenges it presents in parameter tuning.

3.5 Deep Neural Networks

Neural networks have been in the spotlight as AI continues to develop at a rapid pace.

Neural networks are a very useful tool to get impressive predictions. It's a flexible, non-linear prediction method. They can be used for classification and regression. This method takes the inputs and combines them to form hidden variables. We use an activation function like ReLU which can introduce non-linearity. Deep neural networks have multiple hidden layers. They are highly flexible, so we should be cautious about overfitting. As there are a lot of parameters, it's computationally very costly. The power of prediction of this method is huge, however, the interpretability of the parameters becomes very low when the hidden layer is far from the inputs. The number of tuning parameters is large and consequently the set of possible choices is huge. There are the number of hidden layers (depth), the number of neurons of each layer (width), the activation function, the learning rate, the loss function, different forms of regularization (early stopping, dropout), the size of the batch and the number of epochs. There are even more than those mentioned. It's impossible to test every combination of tuning parameters. Therefore, we must fix some and do cross-validation to determine which ones are the best choices in terms of precision, robustness and complexity.



In our case, we had the best predictions with the following method. First, we decided to implement a deep neural network with 3 hidden layers, the ReLU activation function, no regularization and a mean squared error loss function. The score we obtained was not as good as we expected (49.04 in Kaggle). We can see in the graph below that the training MAE is decreasing until the end of the epochs as the validation MAE stops decreasing very early

and remains steady or even begins to increase again. This is not a good sign. The model improves its score for the training set while staying constant for the validation set, it suggests overfitting.

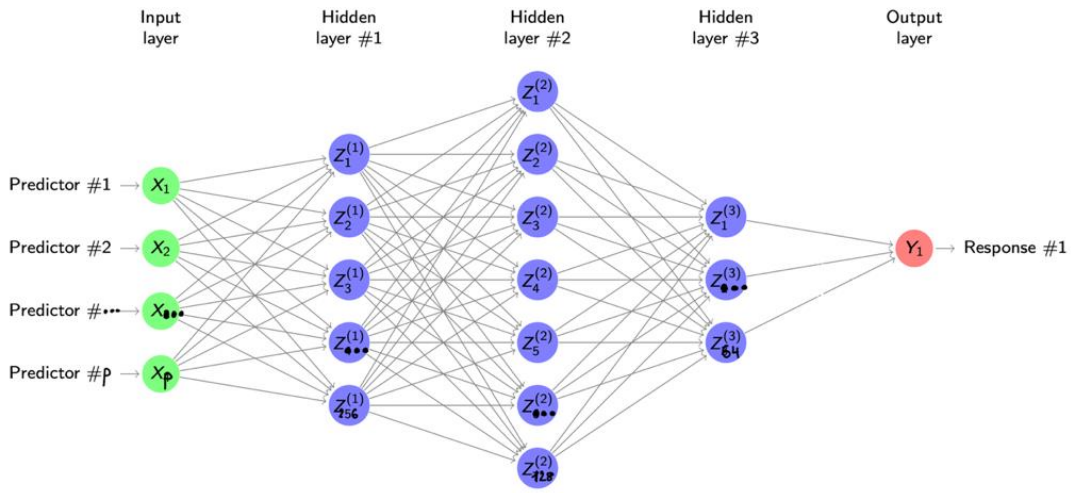
Then, we decided to do cross validation with 5 folds to see which structure is the best to have a comprise between complexity, precision and robustness. We have compared 3 different structures, one with 2 hidden layers of width 64 and 128, one with 3 hidden layers of width 64, 128 and 256 and one with 4 hidden layers of width 64, 128, 256, 128. Results are shown below, and we can highlight that the model with 4 layers is the best one in terms of mean absolute training error and in terms of mean absolute validation error. The better performance in mean absolute training error doesn't necessarily result in a better mean absolute validation error because overfitting might appear. The difference with the model of 3 layers in the validation set is not significant and the standard deviation of the mean absolute validation error is considerably lower, so we prefer less complexity and stays with 3 hidden layers.

Architecture	Mean Train Loss	Std Train Loss	Mean Train MAE	Std Train MAE	Mean Val Loss	Std Val Loss	Mean Val MAE	Std Val MAE
2_layers	1503.48	43.21	25.61	0.30	1801.20	125.92	27.56	0.50
3_layers	1069.28	24.89	21.49	0.37	1615.65	117.91	25.23	0.23
4_layers	915.01	77.68	20.02	0.67	1628.89	87.05	25.18	0.51

[5]

The architecture of our model can be represented like the image below. We have 14 different predictors which are the input layer, then the first hidden layer with 256 neurons, next we have the second hidden layer with 128 neurons, then the third hidden layer with 64 neurons and finally the output layer.

[5] *The style and format of the table was generated with the assistance of ChatGPT. The data and values presented in the table were independently provided and are not derived from ChatGPT.*



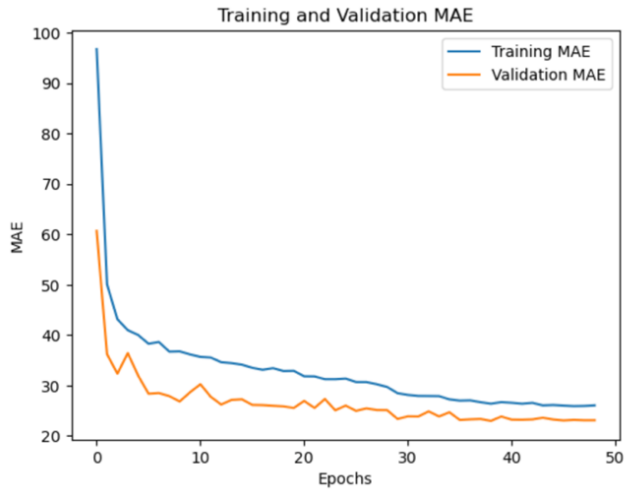
[6]

The squared-error loss has not a closed form solution and is non-convex. With as many parameters as there are in deep neural networks, we are not interested in the minimum of this loss function, because it would lead to overfitting. Taking that into account, we decided to prevent overfitting by early stopping and regularization. Early stopping is useful to return the values of the parameters where the validation error was at the minimum and then, stop the descent. The other type of regularization we decided to implement is the dropout. Dropout is a method where in each update step in the gradient descent, the network structure is changed. In fact, in each step, a proportion of neurons is temporarily randomly disactivated. It forces the model not to be dependent on some neurons and it helps for generalization. By independent research, we also found it interesting to include the batch normalization in the neural network to improve the predictions. It's a useful tool used to make training of the neural networks faster and more stable. It normalizes the mean to 0 and the variance to 1 of the activations. The score we obtained with the regularization and the batch normalization is 38.17 in Kaggle, but it was not enough for us. We thought it was possible to improve the model.

By independent research, we found that changing the activation function could be a possibility to increase the prediction performance. LeakyReLU is a variation of the ReLU function which allows small non-zero gradient for negative inputs, increasing the number of neurons activated. It allowed to further improve the performance of the model. By implementing the LeakyReLU activation function, we end up with a more suitable model with a significantly lower score when testing it in Kaggle (35.75). However, something unexpected

[6] *Modified version of page 2 of the lecture 22 on Deep neural networks in Machine Learning (S403011) course.*

happened. The mean absolute error of the validation set was significantly lower than the mean absolute error of the training set as we can see below.



The two mean absolute errors don't converge to the same value. The model tends to adjust better to the validation than the training. As the distributions of the training and validation are identical, the explication cannot come from there. We didn't manage to do cross validation on the regularization methods. But as we managed to significantly reduce the errors on the predictions we had to make, we decided to keep this model because it is more efficient.

No further improvements were successful. Possible improvements would have been doing cross validation on more tuning parameters to reach a more optimal model. The optimality of the tuning parameters can vary depending on which type of data we have. In our case, we are stuck, because the mean absolute training error is larger than the mean absolute validation error, so we don't really know if the model is performing well or is overfitting.

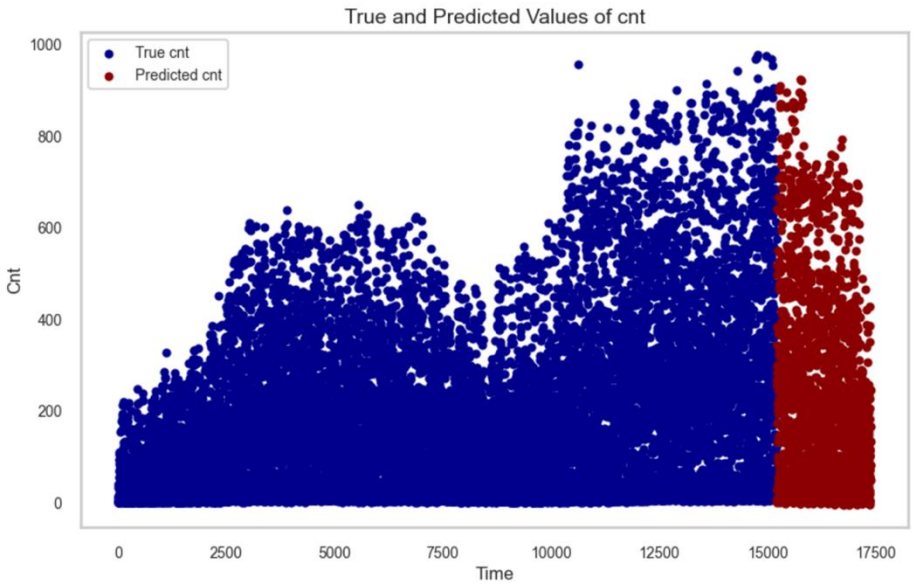
4. Results and Conclusion

We have learnt a lot about different phenomena, and we will now reach conclusions for Mule to help them in their business.

Predictors have different degrees of importance in predicting the number of bikes rented as highlighted in Lasso and Decision Trees different models. Mule must be aware of it and needs to increase their offer in peak times. The main predictors are the time, the temperature and if the day is off or not. It's really aligned with our first insights, as seen in the exploratory data analysis, there are 2 distinct patterns for weekdays and weekend. The peak hours are

completely different, temperature is a key factor for customers and impacts their behavior. We would advise Mule to always stay alert for most of the features but especially for those cited as their impacts are significantly larger than the other ones.

Another important factor is the season, as temperatures can differ a lot, it automatically impacts the number of rentals. As seen in the plot below, the blue points are the true number of bikes rented and we can easily see the seasonal trend and in Winter, the demand is considerably lower. In red, we had to predict data in Fall and Winter for the year 2012, as expected it follows the same seasonal trend as the number of bikes rented decreases, but Mule has great chances to develop well as the dataset from the similar company has developed very well and it significantly increased their number of rentals during those 2 years. Even though the temperature is lower and Fall and Winter are "bad" seasons for their business, predictions are substantially higher than the last Fall and Winter 2011.



To conclude, Mule will probably have more and more customers and the company is likely to develop fast as they could potentially have a lot more bikes rented than in 2011. They need to be aware of which key features are important for their business and bring innovative and effective ideas to support their growth. The future may be bright.

5. References

Bike Sharing ML-Unige Competition: https://www.kaggle.com/competitions/ml-unige-fall-2024-bike-sharing/overview

We have used ChatGPT to overcome coding issues (errors, import…): https://chat.openai.com/chat

Lectures and Seminars from the course.